Project - High Level Design

on

Real Estate Support Triage Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Aarti Bamanke	EN22CS301001
2.	Akshat Joshi	EN22CS304006
3.	Payal Singh	EN22CS301701
4.	Punit Pawar	EN22CS303038
5.	Samruddhi Muley	EN22CS304056
6.	Shubhi Dubey	EN22CS304061

Group Name: 01D11

Project Number: AAI-27

Industry Mentor Name:

University Mentor Name: Prof. Nishant Shrivastava

Academic Year: Jan-Jun 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

Introduction

The Real Estate Triage Agent is an AI-powered support automation system designed to improve the handling of customer communications in the real estate domain. Real estate companies receive a large number of incoming messages such as maintenance complaints, booking requests, rent-related queries, and emergency issues. Manually reviewing and prioritizing these requests can be time-consuming, inconsistent, and inefficient.

To solve this problem, this project uses Large Language Models (LLMs), Agentic AI concepts, and full-stack web technologies to automatically analyze incoming messages, determine urgency and intent, extract key details, generate draft responses, and route requests to the appropriate support team. The system also stores reports in a database and provides a dashboard for monitoring and managing all triage requests.

1.1 Scope of Document

This document provides a complete overview of the Real Estate Triage Agent project, including its objectives, architecture, implementation methodology, modules, technologies used, database integration, dashboard features, testing, and expected outcomes. It explains how artificial intelligence is integrated into a real-world support workflow and how the system can be used to automate message triage in the real estate industry.

The scope of this document also includes frontend-backend communication, report generation, database storage, and dashboard analytics for support operations.

1.2 Intended Audience

This document is intended for the following audience:

- Faculty members and project evaluators reviewing the academic project.
- Students and developers interested in AI-based automation systems.
- Real estate businesses seeking intelligent support management solutions.
- Technical professionals who want to understand the implementation of Agentic AI with LLMs.
- Future developers who may extend or maintain this project.

1.3 System Overview

The Real Estate Triage Agent is a full-stack intelligent system that processes customer support messages in real time. When a user submits a message through the web interface, the backend API receives it and sends it to an AI triage pipeline.

The system uses a ReAct-based LangChain agent connected to a Groq LLM. The agent uses three main tools:

1. **Classifier Tool** – Determines urgency (urgent, medium, low) and intent (complaint, query, booking).
2. **NER Tool** – Extracts important entities such as names, property IDs, phone numbers, dates, and locations.
3. **Response Generator Tool** – Generates a professional draft response based on the analyzed message.

After processing, the system creates a structured triage report containing metadata, classification, extracted entities, routing decisions, SLA response time, and draft response. The report is stored in a database and can be viewed through an interactive dashboard where administrators can filter, review, and manage all reports.

This system helps reduce manual workload, improve response speed, and prioritize urgent issues efficiently.

System Design

2.1 Application Design

The Real Estate Triage Agent is designed using a modular and layered application architecture to ensure scalability, maintainability, and efficient processing of customer support requests. The system integrates a web-based user interface, backend API services, AI-powered processing modules, and database storage into a unified application.

At the presentation layer, the frontend is developed using HTML, CSS, and JavaScript. This layer provides an interactive interface where users can submit real estate support messages, view generated triage reports, and access the reports dashboard for monitoring and management purposes. The frontend communicates with the backend through REST API calls.

The application layer is implemented using Flask, which acts as the backend server. It receives incoming requests from the frontend, validates user input, and routes the data to the AI processing pipeline. After processing, the backend returns structured JSON responses to the frontend and also manages database operations.

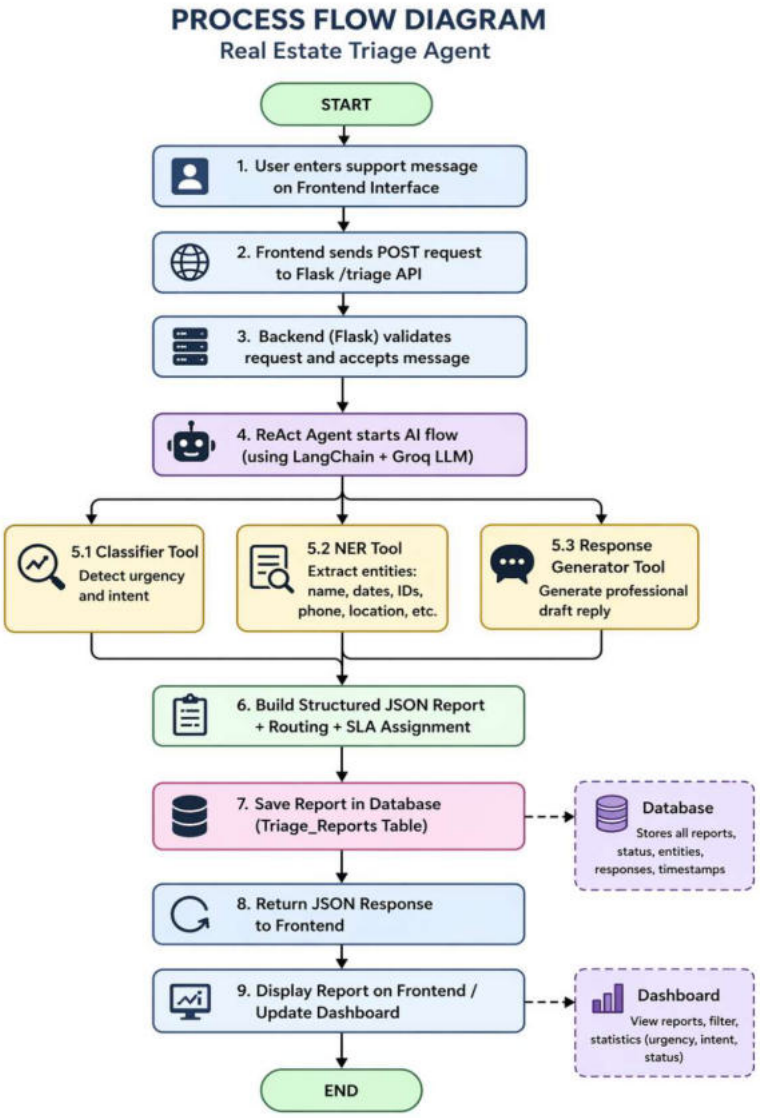
The intelligent processing layer consists of an Agentic AI workflow built using LangChain's ReAct framework and a Groq Large Language Model. This layer uses three specialized tools: a Classifier Tool to determine urgency and intent, a Named Entity Recognition tool to extract important details such as names, dates, and property IDs, and a Response Generator Tool to create professional draft replies. These tools work together to automate the triage process.

The data layer is managed using SQLAlchemy with a relational database. All generated triage reports are stored persistently, including message content, classification results, extracted entities, routing details, response drafts, timestamps, and status information. This allows historical tracking, filtering, and report management.

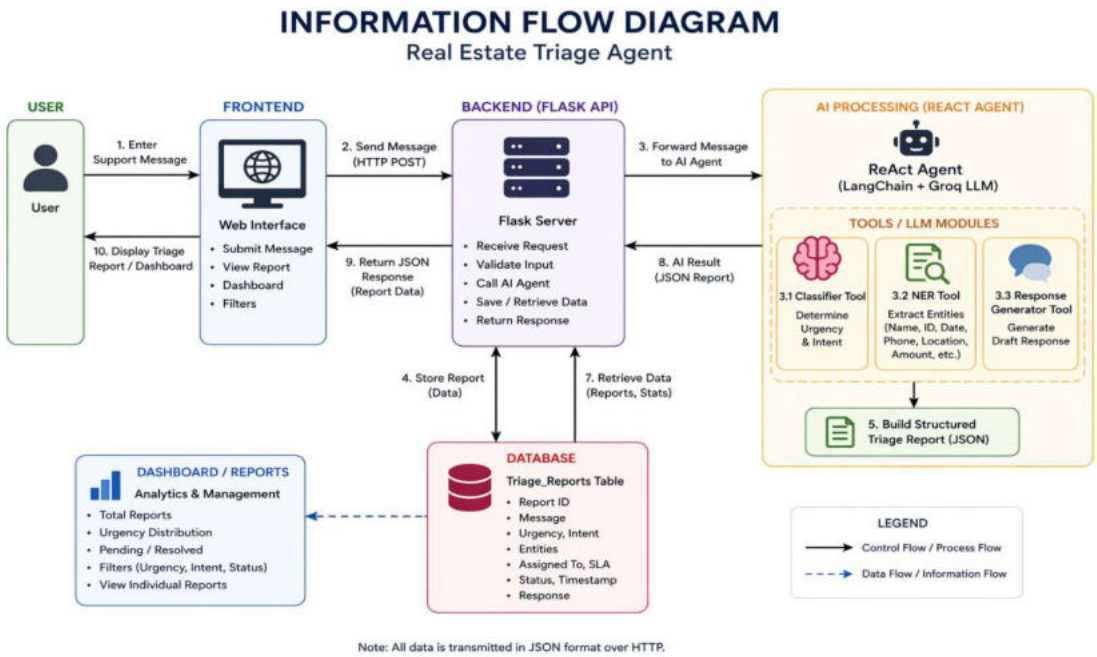
Additionally, a dashboard module is integrated into the application to display report statistics such as total requests, urgency distribution, pending reviews, and resolved cases. Filters are provided to search reports based on urgency, intent, and status.

Overall, the application design follows a structured full-stack approach where each layer performs a dedicated responsibility, resulting in a reliable and efficient intelligent support management system.

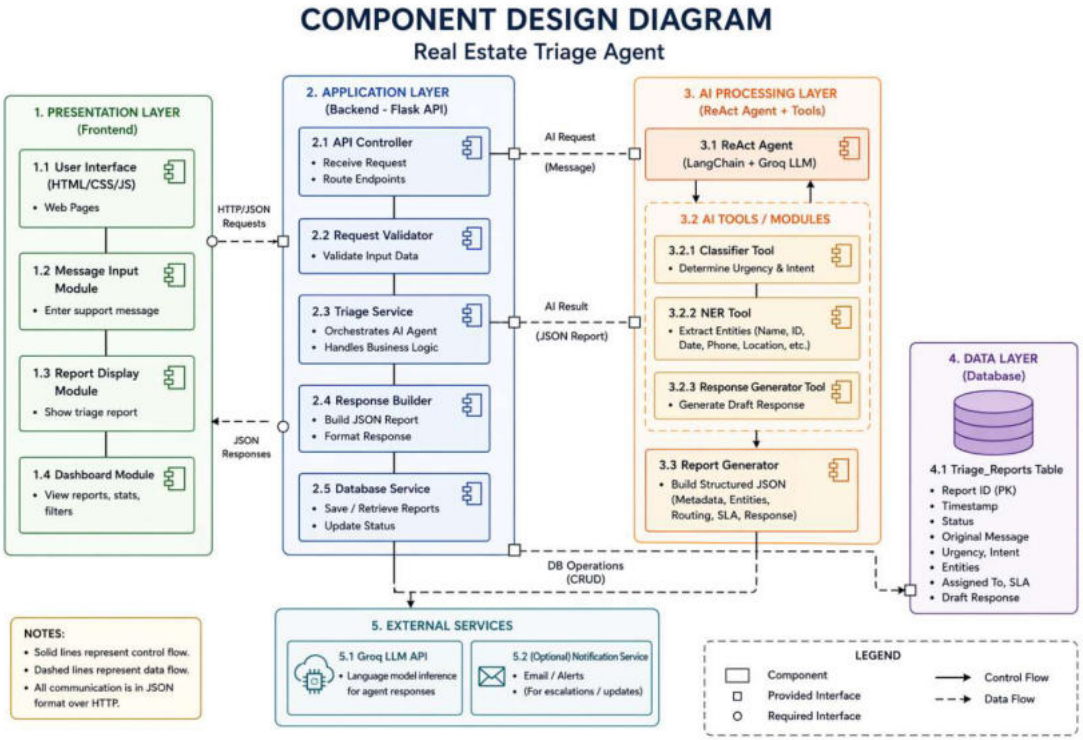
2.2 Process Flow



2.3 Information flow



2.4 Component Design



2.5 Key Design Considerations

The Real Estate Triage Agent was designed with focus on scalability, modularity, performance, and usability. Since the system handles support messages in real time, fast response processing and reliable communication were important requirements.

A modular architecture was used by separating the system into frontend, backend API, AI processing, and database layers. This improves maintainability, testing, and future expansion. The AI module was further divided into three tools: classification, entity extraction, and response generation, making the system more flexible and easier to debug.

LangChain's ReAct agent was selected to intelligently coordinate these tools, while Groq LLM was used for fast inference and real-time processing. Structured JSON outputs were used for smooth data exchange between modules.

Database integration was added to store reports for tracking, filtering, and dashboard analytics. The frontend was designed to be user-friendly with report viewing, filters, and loading/error handling features.

Security was maintained by storing API keys in environment variables, and the codebase was organized into separate folders for clean project structure.

2.6 API Catalogue

The Real Estate Triage Agent uses RESTful APIs developed using Flask to enable communication between the frontend, backend, AI modules, and dashboard system.

API Endpoint	Method	Description	Input	Output
`/`	GET	Health check endpoint to verify server status	None	JSON status message
`/triage`	POST	Accepts a support message and generates a triage report	JSON `{message}`	Full triage report in JSON format
`/reports`	GET	Retrieves all saved reports from database	Optional filters	List of reports
`/reports/<report_id>`	GET	Retrieves a specific report by report ID	report_id in URL	Single report details
`/reports/<report_id>/status`	PUT / PATCH	Updates report review status	JSON `{status}`	Updated report
`/stats`	GET	Returns dashboard statistics	None	Counts by urgency, intent, status

API Details

<u>1. Health Check API</u>

- Endpoint: /
- Method: GET
- Purpose: Confirms backend server is active.

Sample Response:

```
{
 "status": "Real Estate Triage Agent is running"
}
```

2. Triage Processing API

- Endpoint: /triage
- Method: POST
- Purpose: Main AI processing endpoint.

Sample Request:

```
{
 "message": "Water leakage in Flat APT-2047. Please help urgently."
}
```

SampleResponse:
Returns classification, entities, routing, SLA, and generated response.

3. Reports API

- Endpoint: /reports
- Method: GET
- Purpose: Fetches all stored reports for dashboard display.

4. Status Update API

- Endpoint: /reports/<report_id>/status
- Method: PUT / PATCH
- Purpose: Marks report as pending_review, escalated, or resolved.

5. Statistics API

- Endpoint: /stats
- Method: GET
- Purpose: Used by dashboard to show analytics such as total reports, urgency counts, and pending/resolved reports.

Data Design

3.1 Data Model

The system uses a primary table named triage_reports to store all generated reports.

Main Fields:

- id – Auto-increment primary key
- report_id – Unique report identifier
- timestamp – Date and time of report creation
- status – Current status (pending_review, escalated, resolved)
- original_message – Customer support message
- urgency – urgent / medium / low
- intent – complaint / query / booking
- property_ids – Extracted property IDs
- names – Extracted customer names
- dates – Mentioned dates
- phone_numbers – Contact numbers
- locations – Locations mentioned
- amounts – Monetary values
- assigned_to – Assigned support team
- sla – Response deadline
- draft_response – Generated AI reply

3.2 Data Access Mechanism

The system uses SQLAlchemy ORM for database interaction. This allows Python objects to be mapped to database tables and simplifies CRUD operations.

Main operations include:

- Saving new reports
- Fetching all reports
- Retrieving reports by ID

- Updating report status
- Generating dashboard statistics

REST APIs are used by the frontend to access backend data.

3.3 Data Retention Policies

All triage reports are stored in the database for historical tracking and dashboard analytics. Data may be retained until manually deleted or archived by administrators.

Sensitive information such as phone numbers and customer messages should be protected using access control and secure deployment practices. Regular database backups are recommended.

3.4 Data Migration

The system supports data migration through SQLAlchemy table creation and schema updates. Existing JSON report files can also be converted and imported into the database if required.

Future migration can include moving from SQLite to enterprise databases such as MySQL or PostgreSQL for larger scale deployment.

Interfaces

1. User Interface

How users interact with the system.

Include:

- Web-based frontend
- Input textbox for support messages
- Run Triage button
- Report display panel
- Dashboard page
- Filters and status management

2. Frontend–Backend Interface

How frontend talks to backend.

Include:

- HTTP requests using Fetch API
- REST APIs
- JSON request/response format
- Example:
  /triage, /reports, /stats

3. Backend–AI Interface

How Flask connects to AI modules.

Include:

- Flask sends message to ReAct agent
- Agent calls Classifier Tool
- NER Tool

- Response Generator Tool

4. Backend–Database Interface

How backend stores data.

Include:

- SQLAlchemy ORM
- Save reports
- Retrieve reports
- Update status

5. External Interface

Third-party systems used.

Include:

- Groq LLM API
- Environment variables for API key

Caching

Caching is used to improve system performance by temporarily storing frequently accessed or repeated data so that the system can respond faster and reduce unnecessary processing. In the Real Estate Triage Agent, caching can help minimize repeated API calls, reduce database load, and improve user experience.

At the frontend level, browser caching can be used for static files such as HTML, CSS, JavaScript, and images. This reduces page load time and improves responsiveness for returning users.

At the backend level, frequently requested dashboard statistics and report lists can be cached temporarily to reduce repeated database queries. This is especially useful when multiple users access the dashboard simultaneously.

Caching can also be applied to repeated AI requests. If the same support message is submitted multiple times, previously generated results may be reused instead of calling the LLM again, which helps reduce API usage and response time.

For larger-scale deployment, caching tools such as Redis or Memcached can be integrated to provide fast in-memory storage. Appropriate cache expiration policies should be used to ensure updated and accurate information.

Overall, caching helps increase efficiency, reduce latency, and improve scalability of the Real Estate Triage Agent system.

Non-Functional Requirements

7.1 Security Aspects

Security is an important requirement because the system handles customer messages, contact details, and support records. API keys for external services are stored securely using environment variables instead of hardcoding them in the source code.

Input validation is implemented in the backend to prevent invalid or empty requests. Database access is controlled through the backend APIs rather than direct user access. Sensitive data such as phone numbers and customer messages should be protected through secure hosting and access control mechanisms.

For future enhancements, authentication, role-based access control, HTTPS communication, and encrypted database storage can be added for stronger security.

7.2 Performance Aspect

The system is designed for fast response and real-time triage processing. Groq LLM was selected because of its low latency and efficient inference speed. Flask provides lightweight backend processing, while the frontend uses asynchronous API calls for smooth user interaction.

Database storage allows quick retrieval of past reports and dashboard statistics. Caching can be added to further improve performance for repeated requests and analytics queries.

The modular architecture also supports scalability, allowing components such as frontend, backend, and database to be optimized independently.

References

1. LangChain Official Documentation – https://python.langchain.com
2. Groq API Documentation – https://console.groq.com/docs
3. Flask Official Documentation – https://flask.palletsprojects.com
4. SQLAlchemy Documentation – https://www.sqlalchemy.org
5. MDN Web Docs – JavaScript Fetch API – https://developer.mozilla.org
6. Python Official Documentation – https://docs.python.org
7. HTML, CSS, and JavaScript standard web development resources.